Auditoría de seguridad de LabReserva API

Curso: Diseño de Software

José Andrés Acuña Garay - 2023213322

A. MODELADO DE AMENAZAS

El propósito del modelado es identificar qué activos deben protegerse, cuáles son los límites de confianza y qué podría salir mal antes de conocer los detalles internos de la implementación. LabReserva es un sistema para reservar laboratorios de la escuela y contempla estudiantes, profesores y administradores con distintos permisos.

1. Activos que se deben proteger

- Cuentas de usuario: correo, nombre, carnet, teléfono, rol y credenciales.
- Tokens de autenticación y datos utilizados para validar la identidad.
- Información de laboratorios, edificios, comentarios y manuales descargables.
- Reservas: propietario, laboratorio, fecha, horario y propósito.
- Funciones administrativas: creación de laboratorios, consulta global de reservas y eliminación de usuarios.
- Disponibilidad e integridad del servicio, así como la configuración interna necesaria para operarlo.

2. Diagrama de flujo de datos (DFD)

El siguiente DFD representa el sistema desde la perspectiva funcional descrita en el README. El navegador, una aplicación cliente o una herramienta como curl/Postman se consideran externos y no confiables. La API constituye el punto central de validación y aplica las reglas de negocio antes de acceder a los datos o archivos.

Elemento	Descripción y datos intercambiados
Entidad externa: usuario	Estudiante, profesor o administrador. Utiliza navegador, app o cliente HTTP.
Proceso: cliente web / REST Client	Envía solicitudes HTTP, datos JSON y, cuando corresponde, un token Bearer. Recibe respuestas JSON o archivos.
Proceso: API Express	Autentica al usuario, valida entradas, aplica permisos y reglas de negocio y coordina el acceso a recursos.
Almacén: SQLite en memoria	Contiene usuarios, roles, laboratorios, comentarios y reservas.
Almacén: archivos locales	Contiene los manuales de laboratorio que la aplicación permite descargar.
Configuración del servidor	Variables de entorno y secretos requeridos para iniciar y proteger la API.

Flujos principales:

- F1. Usuario → API: solicitudes HTTP con parámetros, datos JSON y token Bearer cuando el endpoint requiere autenticación.
- F2. API → usuario: respuestas JSON con datos, mensajes de error o resultados de operaciones.
- F3. API ↔ SQLite: consultas y modificaciones de usuarios, laboratorios, comentarios y reservas.
- F4. API → archivos locales: lectura de manuales solicitados mediante el endpoint de archivos.
- F5. Configuración → API: carga de variables y secretos al iniciar el servidor.

Límites de confianza identificados:

- LC1. Cliente externo / Internet → API: todo dato recibido puede ser manipulado; el token debe verificarse y los permisos no deben confiarse al cliente.
- LC2. API → SQLite: la API debe limitar cada consulta al usuario y rol autorizados y tratar los parámetros como datos, no como instrucciones SQL.
- LC3. API → sistema de archivos: una solicitud de descarga solo debe permitir archivos dentro del directorio de manuales.
- LC4. Configuración interna → respuestas externas: los secretos y detalles internos no deben exponerse a usuarios.

3. Hipótesis de amenazas STRIDE

Las siguientes hipótesis se plantearon a partir de las funcionalidades documentadas en el README. En esta etapa son escenarios posibles, no afirmaciones de que la vulnerabilidad exista. La confirmación o descarte se realiza en la Parte B.

ID	STRIDE	Superficie	Hipótesis de ataque	Mitigación propuesta	Prioridad
A01	S – Suplantación	Autenticación	Un atacante podría intentar acceder a una cuenta ajena mediante contraseñas débiles, intentos repetidos o uso indebido de un token.	Contraseñas con hash resistente, JWT firmado y con expiración, y limitación de intentos de inicio de sesión.	Alta
A02	T – Manipulación	Edición del perfil	Un estudiante podría enviar campos adicionales al editar su perfil e intentar cambiar datos que no debería poder modificar, como su rol.	Lista blanca de campos editables y validación del lado del servidor.	Crítica
A03	R – Repudio	Operaciones sensibles	Un usuario podría negar haber realizado una acción sensible, como modificar datos o intentar una operación administrativa, si no existe registro suficiente.	Registro de auditoría con usuario, acción, fecha y resultado; nunca registrar contraseñas ni tokens.	Media
A04	I – Divulgación	Directorio de usuarios	Un estudiante podría consultar información de otros usuarios o datos internos que deberían estar reservados a profesores o administradores.	Control de acceso por rol y respuesta limitada a campos públicos.	Alta
A05	I – Divulgación	Consulta de reservas	Un usuario podría cambiar el identificador de una reserva en la URL y consultar una reserva perteneciente a otra persona.	Comprobar la propiedad de la reserva en cada consulta; responder como recurso no disponible para ese usuario.	Alta

A06	T – Manipulación	Cancelación de reservas	Un usuario podría cancelar una reserva ajena si el sistema verifica solamente que el usuario inició sesión, sin comprobar quién es su propietario.	Aplicar la condición de propietario también en la operación de eliminación/cancelación.	Alta
A07	I – Divulgación	Descarga de manuales	Un atacante podría manipular el nombre del archivo solicitado para intentar leer archivos fuera del directorio público de manuales.	Lista permitida de nombres y confinamiento de rutas al directorio autorizado.	Crítica
A08	T – Manipulación	Búsqueda de laboratorios	Un atacante podría introducir caracteres o expresiones especiales en el parámetro de búsqueda para alterar la consulta de datos.	Consultas SQL parametrizadas y validación del parámetro de búsqueda.	Crítica
A09	D – Denegación de servicio	Login y solicitudes HTTP	Un atacante podría enviar solicitudes repetidas o cuerpos de gran tamaño para consumir recursos e impedir el uso normal del sistema.	Límite de intentos, límite de tamaño de solicitudes y controles de abuso.	Media
A10	E – Elevación de privilegios	Funciones administrativas	Un estudiante podría intentar ejecutar operaciones reservadas al administrador, como eliminar usuarios o crear laboratorios.	Autorización por rol en cada endpoint administrativo; denegar por defecto.	Crítica
A11	I – Divulgación	Configuración y diagnóstico	Un usuario podría acceder a rutas de diagnóstico o configuración y obtener información interna o secretos del servidor.	No publicar endpoints de depuración con información sensible y mantener secretos fuera de respuestas al cliente.	Crítica
A12	T – Manipulación	Comentarios y frontend	Un usuario podría publicar contenido HTML o JavaScript como comentario para alterar la interfaz o ejecutar código en el navegador de otros usuarios.	Tratar los comentarios como texto, escapar contenido y evitar insertar HTML no confiable directamente.	Alta

4. Priorización inicial

Se utiliza una valoración cualitativa de probabilidad e impacto, ambos con escala de 1 a 3 (bajo = 1, medio = 2, alto = 3). El nivel de riesgo se obtiene multiplicando ambos valores. La puntuación es una estimación inicial basada en el tipo de activo y el posible efecto; debe revisarse cuando se conozcan las condiciones reales.

Amenazas	Probabilidad	Impacto	Riesgo	Justificación resumida
A02, A07, A08, A10, A11	3	3	9 – Crítico	Podrían permitir elevar privilegios, acceder a secretos, manipular consultas o afectar recursos esenciales.
A01, A04, A05, A06, A09, A12	2–3	2–3	4 a 9 – Alto/Crítico	Pueden comprometer cuentas, información personal, reservas, disponibilidad o sesiones de otros usuarios.
A03	2	2	4 – Medio	La ausencia de trazabilidad dificulta investigar y atribuir acciones sensibles.

La prioridad no significa que el ataque esté confirmado. En la Parte B se documenta qué hipótesis se confirmaron, cuáles se observaron mediante inspección del código y cuáles requieren pruebas adicionales.

B. CACERÍA DE VULNERABILIDADES

La cacería se realizó sobre una copia local del proyecto original, siguiendo la indicación del curso de no probar contra otros grupos, la red del TEC ni sistemas reales. La aplicación utiliza Node.js y Express, con SQLite en memoria. La cuenta de prueba entregada en el README es ana@labreserva.test / ana12345, con rol de estudiante.

1. Preparación y método

1. Descomprimir labreserva-api.zip y abrir la carpeta del proyecto en una terminal.
2. Comprobar que se dispone de Node.js 22.13 o superior.
3. Ejecutar npm install y luego npm run dev.
4. Abrir http://localhost:3000 para la interfaz y utilizar http://localhost:3000/api para las rutas.
5. Autenticarse con la cuenta de Ana mediante POST /api/auth/login y conservar el token devuelto para las rutas protegidas.
6. Usar requests.http con la extensión REST Client o una herramienta equivalente para repetir las solicitudes.

7. Reiniciar el servidor cuando sea necesario restaurar los datos iniciales, porque la base de datos vive en memoria.

Solicitud de autenticación de referencia:

POST /api/auth/login | Content-Type: application/json |
{"email":"ana@labreserva.test","password":"ana12345"}

En las pruebas que requieren autenticación se envía Authorization: Bearer <token_de_Ana>. Las respuestas y códigos indicados abajo corresponden a la ejecución local documentada; los hallazgos identificados únicamente por inspección se señalan expresamente.

2. Resumen de resultados

ID	Hallazgo	Riesgo	Estado de comprobación
H01	Escalada de privilegios mediante edición del perfil	Crítica	Confirmada dinámicamente
H02	Eliminación de usuarios sin autorización administrativa	Crítica	Confirmada dinámicamente
H03	Exposición del directorio y hashes de contraseñas	Alta	Confirmada dinámicamente
H04	Lectura de reservas de otros usuarios (IDOR)	Alta	Confirmada dinámicamente
H05	Cancelación de reservas ajenas (IDOR)	Alta	Identificada en código; no se registró una reproducción dinámica independiente
H06	Exposición de secreto mediante endpoint de diagnóstico	Crítica	Confirmada dinámicamente
H07	Path traversal en descarga de archivos	Crítica	Confirmada dinámicamente
H08	Inyección SQL en búsqueda de laboratorios	Crítica	Confirmada dinámicamente
H09	Uso de MD5 para almacenar contraseñas	Alta	Identificada mediante inspección del código
H10	Secreto JWT predecible y token sin expiración adecuada	Alta	Identificada mediante inspección del código
H11	Posible XSS almacenado en comentarios	Alta	Identificada mediante inspección; no se realizó demostración visual en navegador
H12	Enumeración de usuarios y registro de credenciales en logs	Media	Confirmada mediante solicitudes y revisión de logs
H13	Ausencia de límite de intentos de login	Media	Identificada mediante inspección del código
H14	Exposición de detalles internos en errores	Media	Confirmada dinámicamente
H15	CORS permisivo y límite de JSON demasiado alto	Media	Confirmada mediante revisión y pruebas locales

3. Hallazgos y evidencias

H01 – Escalada de privilegios mediante edición del perfil (crítica)

Reproducción: Solicitud: PATCH /api/users/me con el token de Ana y cuerpo JSON {"role":"admin"}.

Evidencia: Evidencia original: la API respondió 200 y el rol de Ana quedó modificado a admin. En la versión corregida, la solicitud se rechaza con 400 y el rol permanece como estudiante.

Causa raíz: Causa raíz: la ruta y el repositorio de usuarios aceptaban campos que el usuario no debía controlar. Archivos originales: src/modules/users/users.routes.js y users.repository.js.

Corrección propuesta: Corrección propuesta: validar una lista permitida de campos editables (name, phone y carnet) y actualizar únicamente esas columnas. Capa: aplicación y repositorio.

H02 – Eliminación de usuarios sin autorización administrativa (crítica)

Reproducción: Solicitud: DELETE /api/admin/users/4 usando el token de Ana.

Evidencia: Evidencia original: respuesta 204 y eliminación de la cuenta de Beto. En la versión corregida, la solicitud responde 403.

Causa raíz: Causa raíz: el endpoint administrativo no aplicaba una comprobación efectiva del rol antes de eliminar. Archivo original: src/modules/admin/admin.routes.js.

Corrección propuesta: Corrección propuesta: exigir el rol admin mediante middleware de autorización antes de ejecutar la operación. Capa: middleware/rutas.

H03 – Exposición del directorio y hashes de contraseñas (alta)

Reproducción: Solicitud: GET /api/users usando el token de Ana.

Evidencia: Evidencia original: respuesta 200 con información de usuarios y el campo password_hash. En la versión corregida, Ana recibe 403; los roles autorizados reciben solo los campos necesarios.

Causa raíz: Causa raíz: la ruta no restringía adecuadamente el acceso y el repositorio devolvía información sensible. Archivos originales: src/modules/users/users.routes.js y users.repository.js.

Corrección propuesta: Corrección propuesta: autorización por rol y selección explícita de columnas públicas, excluyendo hashes y otros datos sensibles. Capa: rutas y repositorio.

H04 – Lectura de reservas ajenas (IDOR, alta)

Reproducción: Solicitud: GET /api/reservations/1 usando el token de Ana.

Evidencia: Evidencia original: respuesta 200 con una reserva cuyo propietario era otro usuario. En la versión corregida, la respuesta es 404.

Causa raíz: Causa raíz: la consulta utilizaba el identificador de reserva sin filtrar también por el usuario autenticado. Archivos originales: src/modules/reservations/reservations.routes.js y repository.js.

Corrección propuesta: Corrección propuesta: obtener el identificador del usuario desde el token verificado y consultar por id de reserva y user_id. Capa: repositorio/dominio.

H05 – Cancelación de reservas ajenas (IDOR, alta)

Reproducción: Prueba propuesta: DELETE /api/reservations/1 con el token de Ana, donde el identificador corresponde a una reserva de otra persona.

Evidencia: Evidencia: la revisión del SQL original mostró que la eliminación se podía realizar por identificador sin incluir el propietario. Este hallazgo se clasificó como identificado en código; no se presenta como una reproducción dinámica independiente.

Causa raíz: Causa raíz: la operación de eliminación no vinculaba la reserva con el usuario autenticado. Archivos originales: src/modules/reservations/reservations.routes.js y repository.js.

Corrección propuesta: Corrección propuesta: ejecutar DELETE con condiciones WHERE id = ? AND user_id = ? y comprobar el resultado. Capa: repositorio.

H06 – Exposición de secreto mediante endpoint de diagnóstico (crítica)

Reproducción: Solicitud: GET /api/debug/config sin token.

Evidencia: Evidencia original: respuesta 200 con una propiedad de configuración que incluía el secreto JWT. El valor del secreto no se reproduce en este documento. En la versión corregida, la ruta responde 404.

Causa raíz: Causa raíz: existía una ruta de diagnóstico que devolvía configuración interna y estaba montada en la API. Archivos originales: src/modules/debug/debug.routes.js y routes.js.

Corrección propuesta: Corrección propuesta: retirar el endpoint y evitar exponer secretos en respuestas. Si un secreto real fue expuesto, debe rotarse. Capa: API/configuración.

H07 – Path traversal en descarga de archivos (crítica)

Reproducción: Solicitud: GET /api/files?name=../../.env desde el directorio local del proyecto.

Evidencia: Evidencia original: respuesta 200 con contenido del archivo .env, incluyendo configuración sensible. En la versión corregida, la solicitud se rechaza con 400.

Causa raíz: Causa raíz: el nombre recibido se utilizaba para construir la ruta del archivo sin confinamiento suficiente. Archivo original: src/modules/files/files.routes.js.

Corrección propuesta: Corrección propuesta: aceptar únicamente nombres de manuales permitidos, resolver la ruta final y comprobar que permanece dentro del directorio autorizado. Capa: acceso a archivos.

H08 – Inyección SQL en búsqueda de laboratorios (crítica)

Reproducción: Solicitud: GET /api/labs?search=%27%20OR%201%3D1%20--

Evidencia: Evidencia original: la búsqueda devolvió los tres laboratorios aunque el texto buscado no correspondía a ellos. En la versión corregida, la consulta devuelve una lista vacía.

Causa raíz: Causa raíz: el parámetro de búsqueda se concatenaba directamente en la sentencia SQL. Archivo original: src/modules/labs/labs.repository.js.

Corrección propuesta: Corrección propuesta: utilizar consultas preparadas con placeholders y pasar el valor como parámetro. Capa: repositorio/datos.

H09 – Almacenamiento de contraseñas con MD5 (alta)

Reproducción: Comprobación: revisar src/lib/password.js y relacionarlo con los hashes expuestos en H03.

Evidencia: Evidencia: la implementación original utilizaba MD5, un hash rápido y no apropiado para almacenar contraseñas. No se necesitó recuperar una contraseña para identificar el problema.

Causa raíz: Causa raíz: elección de un algoritmo criptográfico inadecuado para contraseñas. Archivo original: src/lib/password.js.

Corrección propuesta: Corrección propuesta: utilizar un algoritmo específico para contraseñas, con sal aleatoria individual y parámetros de costo adecuados, por ejemplo scrypt o Argon2id. Capa: criptografía/autenticación.

H10 – Secreto JWT predecible y tokens sin expiración adecuada (alta)

Reproducción: Comprobación: revisar src/config.js y src/lib/jwt.js.

Evidencia: Evidencia: el código original contemplaba un secreto por defecto y generaba tokens sin una expiración adecuada ni restricción explícita del algoritmo.

Causa raíz: Causa raíz: configuración insegura de la firma y ciclo de vida del JWT. Archivos originales: src/config.js y src/lib/jwt.js.

Corrección propuesta: Corrección propuesta: exigir un secreto aleatorio fuerte mediante variable de entorno, fijar HS256 al verificar y establecer expiración corta. Capa: configuración y autenticación.

H11 – Posible XSS almacenado en comentarios (alta)

Reproducción: Prueba propuesta: publicar un comentario con contenido HTML de prueba y revisar cómo se muestra en la interfaz.

Evidencia: Evidencia: la inspección de public/app.js encontró inserción del contenido del comentario mediante innerHTML. No se ejecutó una demostración visual del evento en un navegador, por lo que se reporta como vulnerabilidad identificada por inspección y no como explotación visual confirmada.

Causa raíz: Causa raíz: contenido proporcionado por usuarios se interpreta como HTML al renderizarse. Archivo original: public/app.js.

Corrección propuesta: Corrección propuesta: crear nodos de texto y asignar contenido mediante textContent; no insertar HTML no confiable. Capa: frontend.

H12 – Enumeración de usuarios y registro de credenciales en logs (media)

Reproducción: Prueba: enviar POST /api/auth/login primero con un correo inexistente y luego con un correo válido y contraseña incorrecta.

Evidencia: Evidencia original: las respuestas diferenciaban los casos (404 para correo inexistente y 401 para contraseña incorrecta). La revisión del log mostró que se registraba el cuerpo de la solicitud. La versión corregida utiliza un 401 genérico y evita registrar contraseñas.

Causa raíz: Causa raíz: respuestas y registros de autenticación revelaban información que no era necesaria. Archivo original: src/modules/auth/auth.routes.js.

Corrección propuesta: Corrección propuesta: usar mensajes uniformes para credenciales inválidas y excluir contraseñas, tokens y secretos de los logs. Capa: autenticación y logging.

H13 – Ausencia de limitación de intentos de login (media)

Reproducción: Prueba propuesta: enviar repetidamente credenciales inválidas desde el mismo origen y observar si existe bloqueo temporal.

Evidencia: Evidencia: la inspección del código original no encontró un control explícito de tasa en el endpoint de login. No se atribuye una denegación de servicio real; se identifica la posibilidad de fuerza bruta.

Causa raíz: Causa raíz: endpoint de autenticación sin limitación de intentos. Archivo original: src/modules/auth/auth.routes.js.

Corrección propuesta: Corrección propuesta: aplicar rate limiting por IP y, cuando sea posible, por cuenta, con respuestas 429 y monitoreo. Capa: borde/autenticación.

H14 – Exposición de detalles internos en errores (media)

Reproducción: Solicitud: GET /api/ruta-inexistente.

Evidencia: Evidencia original: la respuesta de error incluía detalles internos como stack trace o ruta. En la versión corregida se entrega un mensaje genérico sin traza.

Causa raíz: Causa raíz: el manejador de errores devolvía información de depuración al cliente. Archivo original: src/middlewares/error-handler.js.

Corrección propuesta: Corrección propuesta: usar un manejador central que registre detalles solo del lado del servidor y entregue mensajes genéricos al cliente. Capa: manejo de excepciones.

H15 – CORS permisivo y límite de JSON demasiado alto (media)

Reproducción: Prueba: enviar una solicitud con Origin externo y revisar el comportamiento de CORS; revisar el límite configurado para JSON.

Evidencia: Evidencia: el código original reflejaba orígenes y permitía cuerpos JSON de hasta 50 MB. La versión corregida restringe orígenes y limita el cuerpo a 10 KB; una solicitud que supera el límite recibe 413.

Causa raíz: Causa raíz: configuración permisiva del middleware CORS y límite de body excesivo. Archivo original: src/app.js.

Corrección propuesta: Corrección propuesta: configurar una lista de orígenes permitidos, añadir headers de seguridad y reducir el límite del body según las necesidades reales. Capa: borde HTTP.

4. Comparación entre las hipótesis de la Parte A y los resultados de la Parte B

Hipótesis	Hallazgo relacionado	Resultado al finalizar la cacería
A01	H10 y H13	Parcialmente confirmada: se identificaron debilidades de JWT y ausencia de límite de intentos; no se reporta toma de cuenta mediante fuerza bruta.
A02	H01	Confirmada: se pudo modificar el rol mediante la edición del perfil.
A03	No confirmada en esta cacería	La falta de trazabilidad completa debe evaluarse con una revisión específica de logs y auditoría. No se declara como vulnerabilidad demostrada.
A04	H03	Confirmada: la cuenta de estudiante pudo consultar el directorio y hashes.
A05	H04	Confirmada: se obtuvo una reserva de otro usuario mediante cambio de identificador.
A06	H05	Identificada por inspección del SQL; falta conservar una reproducción dinámica independiente.
A07	H07	Confirmada: se pudo acceder al archivo .env manipulando la ruta.
A08	H08	Confirmada: la entrada alteró el resultado de la búsqueda SQL.
A09	H13 y H15	Parcialmente confirmada: se identificó falta de rate limit y un límite de cuerpo excesivo; no se ejecutó una prueba de carga.
A10	H02	Confirmada: una cuenta de estudiante pudo ejecutar una operación administrativa.
A11	H06 y H14	Confirmada: se expuso configuración sensible y detalles internos de errores.
A12	H11	Identificada por inspección de la forma de renderizar comentarios; no se completó demostración visual en navegador.